

ESTACIÓN 3 · TERCERA ENTREGA

Banco de Tecnología CEIPA

Aplicación web de tres capas para el registro de movimientos y el control del saldo de una cuenta

Presentación paso a paso del proyecto: arquitectura, cambios frente al código original, reglas del negocio, puesta en marcha de las tres capas, verificación del CRUD y portafolio profesional

ESTUDIANTE	Deibis Zuluaga Baena
PROGRAMA	Ingeniería de Sistemas · Facultad de Ingeniería
NÚCLEO	122618 Fundamentos de Software (FUNSO_2_N_H)
DOCENTE	Simón Peláez Loaiza
INSTITUCIÓN	CEIPA Business School
TECNOLOGÍAS	React · Vite · Tailwind CSS · Python · Flask · Prisma · MySQL 8 · Docker
REPOSITORIO PÚBLICO	github.com/DavisZulu/Estacion_3
PORTAFOLIO	https://davis-zuluaga.onrender.com
FECHA DE ENTREGA	4 de octubre de 2026

Del CRUD de transacciones a una cuenta bancaria con reglas de negocio

El informe presenta la puesta en marcha y la evolución de una aplicación web organizada en tres procesos independientes: una interfaz construida con React, una API REST desarrollada con Flask y una base de datos MySQL que se ejecuta en un contenedor Docker. El punto de partida fue el proyecto entregado por el docente, un CRUD de transacciones que registraba código, tipo, monto e impacto. Ese proyecto resolvía las cuatro operaciones de extremo a extremo, pero sus datos no representaban ninguna operación real: el impacto se digitaba a mano y los tipos de transacción no tenían ningún efecto sobre la información almacenada.

Para enriquecer el ejercicio, el sistema se transformó en el **Banco de Tecnología CEIPA**, una cuenta que registra cuatro tipos de movimiento: las consignaciones y los avances de tarjeta de crédito, que ingresan dinero, y los pagos y las transferencias, que lo retiran. El campo de impacto se reemplazó por el saldo de la cuenta, que se calcula a partir de los movimientos y nunca puede quedar negativo. Se agregaron validaciones de los datos de entrada, respuestas de error precisas, una ruta de verificación del estado del sistema, la fecha de cada movimiento y un script que prueba automáticamente el CRUD completo contra la API. La interfaz se rediseñó para mostrar el saldo disponible, diferenciar las entradas de las salidas y confirmar las eliminaciones. El sistema se levantó paso a paso en un equipo macOS y la entrega se complementa con el portafolio profesional, incluido en el mismo repositorio.

Palabras clave: arquitectura en capas, API REST, operaciones CRUD, validación de datos, reglas de negocio, MySQL, Docker

01 · ALCANCE

Objetivo y componentes de la entrega

OBJETIVO

Levantar, explicar y mejorar el proyecto completo entregado por el docente, de forma que las tres capas funcionen juntas en un equipo de desarrollo y que el sistema represente una operación bancaria con reglas claras: cada movimiento modifica el saldo y ninguna operación puede dejar la cuenta en negativo.

ENTREGABLE

CONTENIDO

Repositorio público	<code>github.com/DavisZulu/Estacion_3</code> : carpetas <code>BACKEND</code> y <code>FRONTEND</code> , diagramas, capturas, el presente informe y la carpeta <code>PORTAFOLIO_WEB</code> .
Documento del sistema	Descripción, arquitectura, cambios realizados, ejecución paso a paso y verificación.
Sistema en funcionamiento	Las tres capas en ejecución, documentadas en las secciones 05 y 06.

Tres procesos independientes que se comunican por red

La aplicación está formada por tres procesos que se encienden, se apagan y se revisan por separado, cada uno en su propia terminal. Lo único que los une es un contrato de comunicación: HTTP con mensajes en formato JSON entre la interfaz y la API, y Prisma entre la API y la base de datos. El navegador nunca se comunica directamente con MySQL.

Tres procesos independientes, cada uno en su propia terminal. El navegador nunca habla con la base de datos: solo con la API.

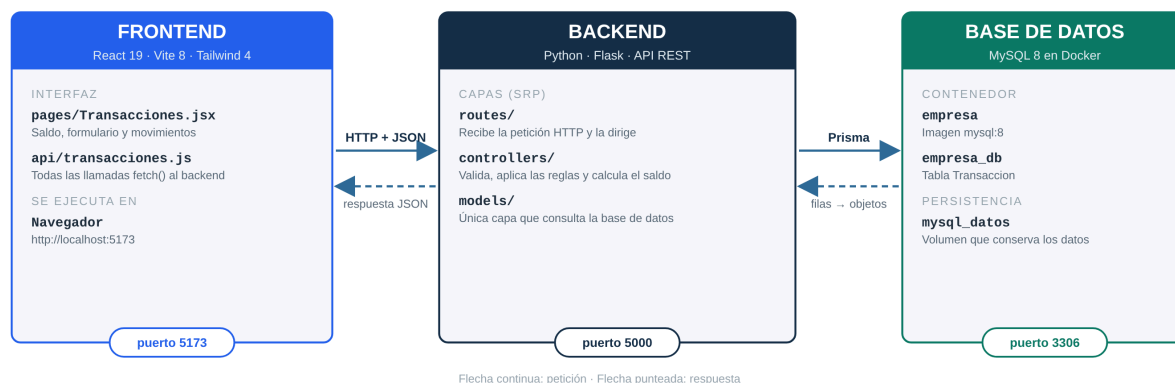


Figura 1. Arquitectura de la aplicación y puertos de cada proceso.

CAPA	TECNOLOGÍA	RESPONSABILIDAD
Frontend	React, Vite, Tailwind CSS	Muestra el saldo, el formulario y la lista de movimientos; envía las peticiones con <code>fetch()</code> y presenta las respuestas. No contiene reglas del negocio.
Backend	Python, Flask, flask-cors	Expone la API REST, valida los datos, aplica las reglas del banco y calcula el saldo.
Acceso a datos	Prisma Client Python	Traduce el modelo definido en <code>schema.prisma</code> a una tabla de MySQL y ejecuta las consultas.
Base de datos	MySQL 8 en Docker	Almacena los movimientos. El volumen <code>mysql_datos</code> conserva los datos aunque el contenedor se elimine.

Las capas del backend

Dentro de la API, cada petición recorre tres carpetas, y cada una tiene una sola responsabilidad. Esta organización permite cambiar una parte sin afectar a las demás: las reglas del banco se agregaron en el controller sin modificar las rutas ni la forma de consultar la base de datos.

CARPETA	RESPONSABILIDAD
routes/	Define la URL y el método HTTP (GET, POST, PUT, DELETE) y entrega la petición al controller. No contiene lógica.
controllers/	Valida los datos, aplica las reglas del banco, calcula el saldo y decide el código de respuesta.
models/	Es la única capa que consulta la base de datos, mediante Prisma.

Punto de partida y cambios realizados

El código original del proyecto registraba transacciones con cuatro datos: código, tipo (crédito, débito o transferencia), monto e impacto. Al ponerlo en marcha se encontró en la base de datos la transacción T001, un crédito por \$400.000 con un impacto de 1.221 (Figura 4). Ese valor había sido digitado a mano y no correspondía a ninguna regla: el sistema lo aceptó porque no tenía forma de saber si era correcto. A partir de esa observación se revisó el resto del código.

ASPECTO	CÓDIGO ORIGINAL	VERSIÓN ENTREGADA
Tipos de movimiento	Crédito, débito y transferencia, sin efecto sobre ningún dato.	Consignar y avance suman al saldo; pagar y transferir restan.
Impacto	Campo digitado a mano por el usuario.	Se elimina. En su lugar se muestra el saldo después de cada movimiento.
Saldo	No existía.	Se calcula a partir de los movimientos y nunca puede quedar negativo.
Validación	Solo rechazaba el monto negativo; aceptaba el cero, un código vacío y cualquier tipo.	Código obligatorio, tipo permitido, monto entero mayor que cero y saldo suficiente.
Eliminar un registro inexistente	Respondía «Transaccion eliminada» aunque no se había eliminado nada.	Responde 404: «La transacción no existe».
Actualizar un registro inexistente	Respondía un error interno de Python.	Responde 404: «La transacción no existe».
Código repetido	Respondía el mensaje técnico de la base de datos.	Responde 409: «Ya existe una transacción con el código...».
Base de datos apagada	Página de error HTML que la interfaz no puede leer.	Responde 503 en JSON con la instrucción para encender el contenedor.
Fecha	No se registraba.	Columna <code>fecha</code> que MySQL completa al crear el movimiento.
Pruebas	Manuales, comando por comando.	El script <code>probar_api.py</code> prueba el CRUD completo y las reglas en una sola ejecución.
Interfaz	Formulario y tabla, sin confirmación al eliminar y sin aviso si el servidor está apagado.	Saldo visible, entradas en verde y salidas en rojo, confirmación al eliminar y aviso de conexión.
Credenciales	El archivo <code>.env</code> con la contraseña formaba parte del repositorio.	<code>.env</code> excluido; se entrega <code>.env.example</code> .

CRITERIO APLICADO

Las reglas del negocio deben vivir en el servidor y no en el formulario. Si el cálculo depende de lo que el usuario escribe, la base de datos termina guardando valores que nadie puede verificar. Por eso el saldo no se digita ni se almacena: el backend lo calcula cada vez a partir de los movimientos registrados.

Reglas del banco y cálculo del saldo

La cuenta empieza en \$0. Cada movimiento suma o resta su monto al saldo según el tipo, y ninguna operación puede dejar la cuenta en negativo.

TIPO	SIGNIFICADO	EFEECTO EN EL SALDO
CONSIGNAR	Ingreso de dinero a la cuenta.	Suma el monto.
AVANCE	Ingreso de dinero desde la tarjeta de crédito.	Suma el monto.
PAGAR	Salida de dinero para pagar un servicio o una compra.	Resta el monto.
TRANSFERIR	Salida de dinero hacia otra cuenta.	Resta el monto.

En el código, la regla se resuelve con un diccionario que indica si cada tipo suma (+1) o resta (-1). Para calcular el saldo se recorren los movimientos del más antiguo al más reciente y a cada uno se le asigna el saldo que quedó después de aplicarlo:

```
# Cada tipo dice si suma (+1) o resta (-1) al saldo.
TIPOS = {
    "CONSIGNAR": +1,
    "AVANCE": +1,
    "PAGAR": -1,
    "TRANSFERIR": -1,
}

def calcular_saldos(movimientos):
    saldo = 0
    resultado = []
    for t in movimientos:
        saldo = saldo + TIPOS[t["tipo"]] * t["monto"]
        resultado.append(**t, "saldo": saldo)
    return resultado
```

Por qué el saldo se calcula y no se guarda

Si el saldo se almacenara como una columna, cada edición o eliminación de un movimiento antiguo obligaría a corregir el saldo de todos los movimientos posteriores, y un error en esa corrección dejaría la cuenta descuadrada. Al calcularlo cada vez a partir de los movimientos, el saldo siempre coincide con lo registrado. La misma función se usa antes de crear, editar o eliminar: se prueba cómo quedaría la cuenta con el cambio y, si en algún momento el saldo baja de cero, la operación se rechaza. La función `crear_transaccion()`, presentada en la página siguiente, muestra ese orden: primero valida, luego prueba el saldo y solo entonces guarda.

Validación, respuestas de error y modelo de clases

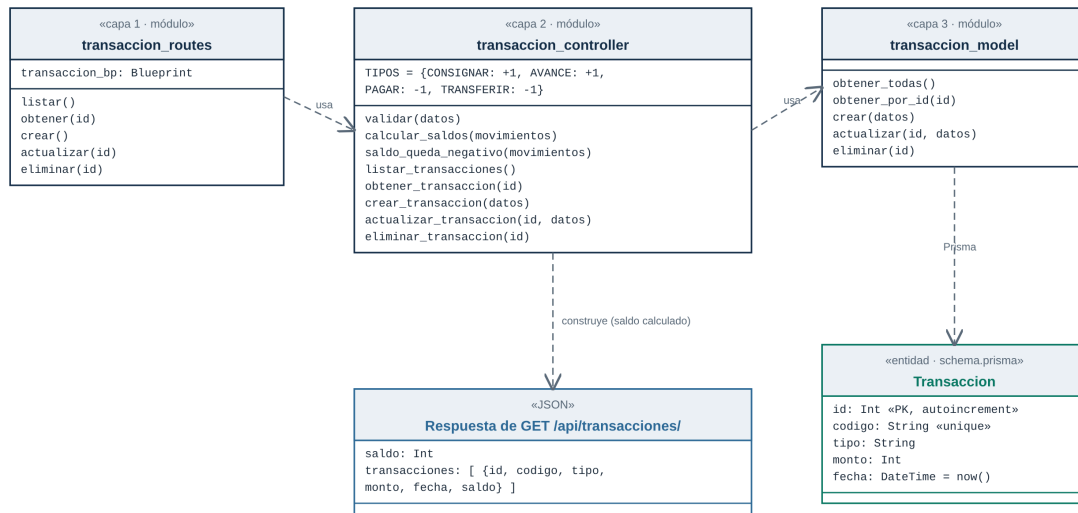
La función `validar()` revisa los datos antes de que lleguen a la base de datos. Si algo no cumple, devuelve un mensaje pensado para el usuario y la API responde con el código HTTP que corresponde.

```
def crear_transaccion(datos):
    limpios, error = validar(datos)
    if error:
        return {"error": error}, 400

    # Se prueba como quedaria la cuenta con el nuevo movimiento al final.
    movimientos = movimientos_actuales() + [limpios]
    if saldo_queda_negativo(movimientos):
        return {"error": "Saldo insuficiente para esta transacción"}, 400

    try:
        nueva = transaccion_model.crear(limpios)
    except UniqueViolationError:
        return {"error": f"Ya existe una transacción con el código {limpios['codigo']}"}, 409
    return a_diccionario(nueva), 201
```

CÓDIGO	CUÁNDO SE RESPONDE	MENSAJE
200 / 201	Operación exitosa / movimiento creado	El movimiento en formato JSON.
400	Datos inválidos o saldo insuficiente	«El código es obligatorio» · «El tipo debe ser CONSIGNAR, AVANCE, PAGAR o TRANSFERIR» · «El monto debe ser mayor que cero» · «Saldo insuficiente para esta transacción»
404	El movimiento no existe	«La transacción no existe»
409	El código ya está registrado	«Ya existe una transacción con el código TX-0001»
503	MySQL no responde	«La base de datos no está disponible. Enciende el contenedor: docker start empresa»



Línea punteada con flecha abierta: dependencia («usa»). El saldo no es una columna: lo calcula el controller.

Figura 2. Diagrama de clases del backend: las tres capas, la entidad **Transaccion** definida en **schema.prisma** y la respuesta JSON con el saldo calculado.

Interfaz del banco

La pantalla muestra en el encabezado el nombre del banco y el saldo disponible. El formulario registra o edita un movimiento con su código, tipo y monto, y la tabla presenta los movimientos del más reciente al más antiguo con el saldo que quedó después de cada uno. Las entradas se muestran en verde con signo positivo y las salidas en rojo con signo negativo. Antes de eliminar, la aplicación pide confirmación; si el backend no responde, aparece un aviso con la opción de reintentar.

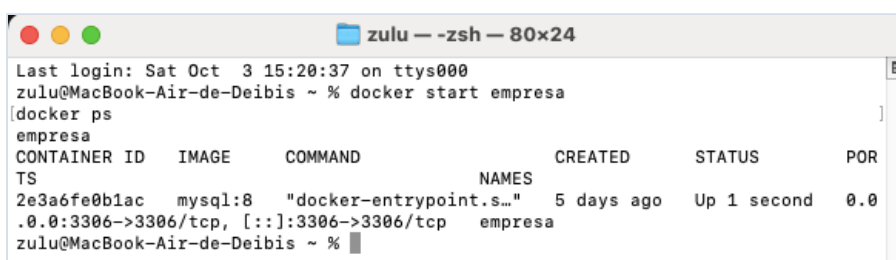
Ejecución de las tres capas, paso a paso

El sistema se levantó en un MacBook Air con macOS, Docker Desktop, Python 3.14 y Node.js. El orden importa: primero la base de datos, después el backend y por último el frontend.

PASO 1 Encender la base de datos

El contenedor `empresa` ya existía, por lo que basta con encenderlo. `docker ps` confirma que está activo y que publica el puerto 3306.

```
docker start empresa
docker ps
```



```

Last login: Sat Oct 3 15:20:37 on ttys000
zulu@MacBook-Air-de-Deibis ~ % docker start empresa
[docker ps
empresa
CONTAINER ID   IMAGE     COMMAND                  CREATED    STATUS    PORTS
2e3a6fe0b1ac   mysql:8   "docker-entrypoint.s..." 5 days ago Up 1 second 0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp
zulu@MacBook-Air-de-Deibis ~ % ]
  
```

Figura 3. El contenedor de MySQL encendido y publicando el puerto 3306.

PASO 2 Comprobar MySQL y revisar los datos existentes

`mysqladmin ping` confirma que MySQL acepta conexiones. La consulta muestra la tabla del código original con la transacción T001 y su impacto digitado a mano.

```
docker exec empresa mysqladmin ping -uroot -p12345
docker exec empresa mysql -uroot -p12345 empresa_db -e "SHOW TABLES; SELECT * FROM Transaccion;"
```

```

Last login: Sat Oct 3 15:20:37 on ttys000
zulu@MacBook-Air-de-Deibis ~ % docker start empresa
[docker ps
empresa
CONTAINER ID   IMAGE     COMMAND                  CREATED    STATUS    PORTS
2e3a6fe0b1ac   mysql:8   "docker-entrypoint.s..." 5 days ago Up 1 second 0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp
zulu@MacBook-Air-de-Deibis ~ % docker exec empresa mysqladmin ping -uroot -p12345
mysqladmin: [Warning] Using a password on the command line interface can be insecure.
mysql is alive
[mysql: [Warning] Using a password on the command line interface can be insecure.
Tables_in_empresa_db
Transaccion
Usuario
id      codigo tipo  monto  impacto
1       T001   CREDITO 400000 1221
zulu@MacBook-Air-de-Deibis ~ % ]
  
```

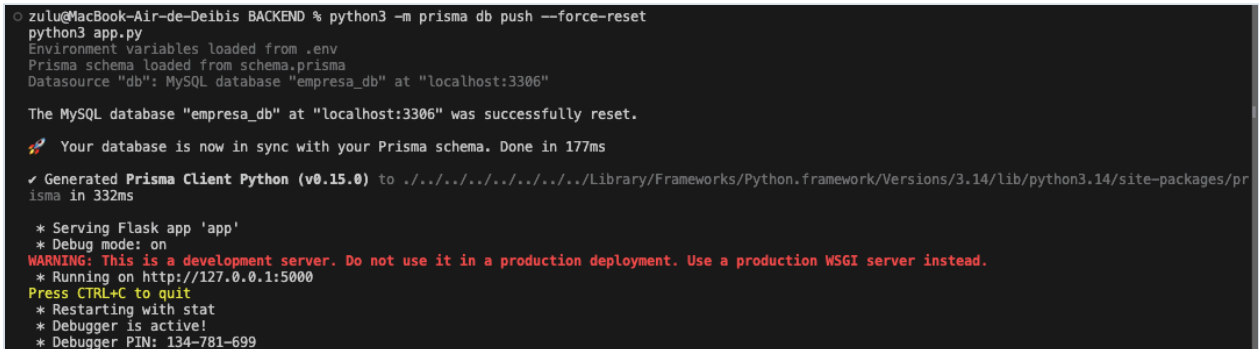
Figura 4. MySQL responde. La transacción T001 del código original registra un impacto de 1.221 digitado a mano.

Backend: nueva estructura de la tabla y API en ejecución

PASO 3 Crear la tabla con la nueva estructura y encender la API

En una segunda terminal, dentro de `BACKEND`, se instalaron las dependencias con `pip3 install -r requirements.txt`. Como la tabla cambió (sale el impacto y entra la fecha) y la cuenta debía empezar en \$0, se ejecutó `prisma db push --force-reset`, que reinicia la base de datos, crea la tabla según `schema.prisma` y genera el cliente de Prisma. A continuación, `python3 app.py` enciende la API en el puerto 5000. Esta terminal debe permanecer abierta.

```
cd ~/Desktop/ProyectosCeipa/Semana_7/Estacion_3/BACKEND
python3 -m prisma db push --force-reset
python3 app.py
```



```
zulu@MacBook-Air-de-Deibis BACKEND % python3 -m prisma db push --force-reset
python3 app.py
Environment variables loaded from .env
Prisma schema loaded from schema.prisma
Datasource "db": MySQL database "empresa_db" at "localhost:3306"

The MySQL database "empresa_db" at "localhost:3306" was successfully reset.

🚀 Your database is now in sync with your Prisma schema. Done in 177ms

✓ Generated Prisma Client Python (v0.15.0) to ../../../../../../Library/Frameworks/Python.framework/Versions/3.14/lib/python3.14/site-packages/prisma in 332ms

* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 134-781-699
```

Figura 5. Base de datos reiniciada y sincronizada con el esquema; la API queda escuchando en `http://127.0.0.1:5000`.

Prueba automática del CRUD y datos de ejemplo

PASO 4 Probar el CRUD completo contra la API

En una tercera terminal se ejecutó `probar_api.py`. El script reemplaza las pruebas manuales: envía peticiones reales a la API, compara el código de respuesta con el esperado y muestra el saldo después de cada operación. Cubre las cuatro operaciones del CRUD y las reglas del banco, y al terminar elimina sus propios movimientos de prueba, de modo que el saldo vuelve a \$0.

```
python3 probar_api.py
```

```
zulu@MacBook-Air-de-Deibis Proyecto_Completo % cd ~/Desktop/ProyectosCeipa/Semana_7/Estacion_3/BACKEND
python3 probar_api.py
Banco de Tecnología CEIPA - prueba automática del CRUD

✓ 0. La API y la base de datos responden HTTP 200 (esperado 200)
  → {'api': 'ok', 'base_datos': 'conectada', 'transacciones': 0}
  → saldo inicial: $0

CREATE
✓ 1. Consignar $1.000.000 HTTP 201 (esperado 201)
  → saldo: $1.000.000
✓ 2. Pagar $300.000 HTTP 201 (esperado 201)
  → saldo: $700.000

READ
✓ 3. Consultar el pago por su id HTTP 200 (esperado 200)
  → PRUEBA-02 - PAGAR - $300.000

UPDATE
✓ 4. Cambiar el pago a $500.000 HTTP 200 (esperado 200)
  → saldo: $500.000

REGLAS DEL BANCO
✓ 5. Transferir más de lo que hay HTTP 400 (esperado 400)
  → Saldo insuficiente para esta transacción
✓ 6. Monto negativo HTTP 400 (esperado 400)
  → El monto debe ser mayor que cero
✓ 7. Tipo que no existe HTTP 400 (esperado 400)
  → El tipo debe ser CONSIGNAR, AVANCE, PAGAR o TRANSFERIR
✓ 8. Código repetido HTTP 409 (esperado 409)
  → Ya existe una transacción con el código PRUEBA-01
✓ 9. Eliminar un id que no existe HTTP 404 (esperado 404)
  → La transacción no existe

DELETE
✓ 10. Eliminar el pago HTTP 200 (esperado 200)
  → Transacción PRUEBA-02 eliminada
✓ 11. Eliminar la consignación HTTP 200 (esperado 200)
  → saldo de vuelta en $0

Resultado: 12/12 pruebas correctas
```

Figura 6. Las doce verificaciones correctas: creación, lectura, actualización y eliminación, saldo insuficiente, monto negativo, tipo inexistente, código repetido y registro inexistente.

PASO 5 Cargar movimientos de ejemplo

Con la opción `--semilla`, el mismo script registra ocho movimientos de ejemplo a través de la API, de modo que cada uno pasa por las mismas validaciones que un movimiento digitado en la interfaz. La cuenta queda con un saldo de \$3.509.000.

```
python3 probar_api.py --semilla
```

```
zulu@MacBook-Air-de-Deibis BACKEND % python3 probar_api.py --semilla

Cargando movimientos de ejemplo

TX-0001 CONSIGNAR $3.500.000 → creado
TX-0002 PAGAR $420.000 → creado
TX-0003 TRANSFERIR $850.000 → creado
TX-0004 AVANCE $1.000.000 → creado
TX-0005 PAGAR $185.000 → creado
TX-0006 CONSIGNAR $1.200.000 → creado
TX-0007 TRANSFERIR $640.000 → creado
TX-0008 PAGAR $96.000 → creado

Saldo disponible: $3.509.000
```

Figura 7. Ocho movimientos de ejemplo registrados y saldo disponible resultante.

Frontend en ejecución

PASO 6 Encender la interfaz

Dentro de `FRONTEND`, `npm install` descarga las dependencias y `npm run dev` levanta el servidor de desarrollo de Vite en el puerto 5173. Al abrir esa dirección en el navegador, la interfaz consulta la API y muestra el saldo y los movimientos almacenados en MySQL.

```
cd ../FRONTEND
npm install
npm run dev
```

```
zulu@MacBook-Air-de-Deibis BACKEND % cd ../FRONTEND
npm install
npm run dev
→ Local: http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

Figura 8. Vite sirve la interfaz en `http://localhost:5173`.

The screenshot shows a web browser at `localhost:5173` displaying the 'Banco de Tecnología CEIPA' interface. The header includes the bank logo, name, and a 'SALDO DISPONIBLE' of \$3,509,000. Below the header is a 'Nueva transacción' form with fields for 'Código' (TX-0009), 'Tipo' (Consignar (+ entra)), and 'Monto (COP)' (500000), with a 'Registrar transacción' button. A 'Movimientos' section displays a table of transactions.

Código	Tipo	Monto	Saldo	Fecha	Acciones
TX-0008	PAGAR	- \$ 96.000	\$ 3.509.000	4/10/2026, 8:39 p. m.	Editar Eliminar
TX-0007	TRANSFERIR	- \$ 640.000	\$ 3.605.000	4/10/2026, 8:39 p. m.	Editar Eliminar
TX-0006	CONSIGNAR	+ \$ 1.200.000	\$ 4.245.000	4/10/2026, 8:39 p. m.	Editar Eliminar
TX-0005	PAGAR	- \$ 185.000	\$ 3.045.000	4/10/2026, 8:39 p. m.	Editar Eliminar
TX-0004	AVANCE	+ \$ 1.000.000	\$ 3.230.000	4/10/2026, 8:39 p. m.	Editar Eliminar
TX-0003	TRANSFERIR	- \$ 850.000	\$ 2.230.000	4/10/2026, 8:39 p. m.	Editar Eliminar
TX-0002	PAGAR	- \$ 420.000	\$ 3.080.000	4/10/2026, 8:39 p. m.	Editar Eliminar
TX-0001	CONSIGNAR	+ \$ 3.500.000	\$ 3.500.000	4/10/2026, 8:39 p. m.	Editar Eliminar

React + Vite + Tailwind - API Flask - MySQL en Docker

Figura 9. Banco de Tecnología CEIPA: saldo disponible, formulario de registro y movimientos con el saldo resultante de cada uno.

Operaciones CRUD desde la interfaz

Las cuatro operaciones se ejecutaron desde el navegador. En cada caso, la interfaz envía la petición a la API, el controller aplica las reglas del banco y la pantalla se actualiza con el nuevo saldo.

Banco de Tecnología CEIPA
Tu cuenta, tus movimientos, en tiempo real

SALDO DISPONIBLE
\$ 3.259.000

Nueva transacción

Código: TX-0009 Tipo: Consignar (+ entra) Monto (COP): 500000

✓ Transacción registrada

Registrar transacción

Movimientos

Código	Tipo	Monto	Saldo	Fecha	Acciones
TX-0009	PAGAR	- \$ 250.000	\$ 3.259.000	4/10/2026, 8:44 p. m.	Editar Eliminar
TX-0008	PAGAR	- \$ 96.000	\$ 3.509.000	4/10/2026, 8:39 p. m.	Editar Eliminar

Figura 10. Create: registro del pago TX-0009 por \$250.000. El saldo baja de \$3.509.000 a \$3.259.000.

Banco de Tecnología CEIPA
Tu cuenta, tus movimientos, en tiempo real

SALDO DISPONIBLE
\$ 3.259.000

Nueva transacción

Código: TX-0010 Tipo: Transferir (- sale) Monto (COP): 5000000

Saldo insuficiente para esta transacción

Registrar transacción

Movimientos

Código	Tipo	Monto	Saldo	Fecha	Acciones
TX-0009	PAGAR	- \$ 250.000	\$ 3.259.000	4/10/2026, 8:44 p. m.	Editar Eliminar
TX-0008	PAGAR	- \$ 96.000	\$ 3.509.000	4/10/2026, 8:39 p. m.	Editar Eliminar

Figura 11. Regla del banco: una transferencia de \$5.000.000 se rechaza porque supera el saldo disponible.

Banco de Tecnología CEIPA
Tu cuenta, tus movimientos, en tiempo real

SALDO DISPONIBLE
\$ 3.209.000

Nueva transacción

Código: TX-0009 Tipo: Consignar (+ entra) Monto (COP): 500000

✓ Transacción actualizada

Registrar transacción

Movimientos

Código	Tipo	Monto	Saldo	Fecha	Acciones
TX-0009	PAGAR	- \$ 300.000	\$ 3.209.000	4/10/2026, 8:44 p. m.	Editar Eliminar
TX-0008	PAGAR	- \$ 96.000	\$ 3.509.000	4/10/2026, 8:39 p. m.	Editar Eliminar

Figura 12. Update: el pago TX-0009 se edita a \$300.000 y el saldo se recalcula en \$3.209.000.

localhost:5173 dice

¿Eliminar la transacción TX-0009 por \$ 300.000?

Cancelar Aceptar

Banco de Tecnología CEIPA
Tu cuenta, tus movimientos, en tiempo real

SALDO DISPONIBLE
\$ 3.209.000

Nueva transacción

Código: TX-0009 Tipo: Consignar (+ entra) Monto (COP): 500000

✓ Transacción actualizada

Registrar transacción

Movimientos

Código	Tipo	Monto	Saldo	Fecha	Acciones
TX-0009	PAGAR	- \$ 300.000	\$ 3.209.000	4/10/2026, 8:44 p. m.	Editar Eliminar
TX-0008	PAGAR	- \$ 96.000	\$ 3.509.000	4/10/2026, 8:39 p. m.	Editar Eliminar

Figura 13. Delete: antes de eliminar, la aplicación pide confirmación con el código y el monto.

Eliminación y persistencia en MySQL

Banco de Tecnología CEIPA
Tu cuenta, tus movimientos, en tiempo real

SALDO DISPONIBLE
\$ 3.509.000

Nueva transacción

Código: TX-0009 Tipo: Consignar (+ entra) Monto (COP): 500000

✓ Transacción TX-0009 eliminada

Registrar transacción

Movimientos

Código	Tipo	Monto	Saldo	Fecha	Acciones
TX-0008	PAGAR	- \$ 96.000	\$ 3.509.000	4/10/2026, 8:39 p. m.	Editar Eliminar
TX-0007	TRANSFERIR	- \$ 640.000	\$ 3.605.000	4/10/2026, 8:39 p. m.	Editar Eliminar
TX-0006	CONSIGNAR	+ \$ 1.200.000	\$ 4.245.000	4/10/2026, 8:39 p. m.	Editar Eliminar
TX-0005	PAGAR	- \$ 185.000	\$ 3.045.000	4/10/2026, 8:39 p. m.	Editar Eliminar
TX-0004	AVANCE	+ \$ 1.000.000	\$ 3.230.000	4/10/2026, 8:39 p. m.	Editar Eliminar
TX-0003	TRANSFERIR	- \$ 850.000	\$ 2.230.000	4/10/2026, 8:39 p. m.	Editar Eliminar
TX-0002	PAGAR	- \$ 420.000	\$ 3.080.000	4/10/2026, 8:39 p. m.	Editar Eliminar
TX-0001	CONSIGNAR	+ \$ 3.500.000	\$ 3.500.000	4/10/2026, 8:39 p. m.	Editar Eliminar

Figura 14. Delete confirmado: TX-0009 se elimina y el saldo vuelve a \$3.509.000. Read: la tabla se vuelve a consultar después de cada operación.

Persistencia: los datos dentro de MySQL

Para comprobar que los movimientos quedan almacenados en la base de datos y no en la memoria de la aplicación, la tabla se consultó directamente en MySQL con la extensión Database Client de Visual Studio Code. Los identificadores comienzan en 4 porque los números 1 a 3 los usó el script de pruebas antes de eliminar sus movimientos: MySQL no reutiliza los valores de una columna `AUTO_INCREMENT`. La fecha se almacena en hora universal (UTC) y la interfaz la presenta en la hora de Colombia.

Database: 127.0.0.1@3306 8.4.11

Security: empresa_db 84K

Query: Tables (2)

Transaccion 8

Usuario

Views

Functions

Procedures

mysql

Transaccion

Properties DATA ER Mock Manager {} empresa_db

SELECT * FROM 'Transaccion' LIMIT 100

Search Results

Cost: 82ms < 1 > Total 8

	id	código	tipo	monto	fecha
	int	varchar(191)	varchar(191)	int	datetime(3)
>	4	TX-0001	CONSIGNAR	3500000	2026-10-05 01:39:42.593
>	5	TX-0002	PAGAR	420000	2026-10-05 01:39:42.669
>	6	TX-0003	TRANSFERIR	850000	2026-10-05 01:39:42.782
>	7	TX-0004	AVANCE	1000000	2026-10-05 01:39:42.830
>	8	TX-0005	PAGAR	185000	2026-10-05 01:39:43.006
>	9	TX-0006	CONSIGNAR	1200000	2026-10-05 01:39:43.037
>	10	TX-0007	TRANSFERIR	640000	2026-10-05 01:39:43.201
>	11	TX-0008	PAGAR	96000	2026-10-05 01:39:43.240

Figura 15. La tabla `Transaccion` en MySQL, consultada desde Visual Studio Code: ocho movimientos con código, tipo, monto y fecha.

Comportamiento ante una caída de la base de datos

Para comprobar qué ocurre si la base de datos deja de responder, se detuvo el contenedor de MySQL con la aplicación en uso. La API no se detiene: el manejador de errores responde con el código 503 y un mensaje en formato JSON, y la interfaz lo muestra con la opción de reintentar. Al encender de nuevo el contenedor y reintentar, el sistema recupera el saldo y los movimientos sin reiniciar el backend ni el frontend.

```
docker stop empresa      # se apaga la base de datos
docker start empresa     # se vuelve a encender
```

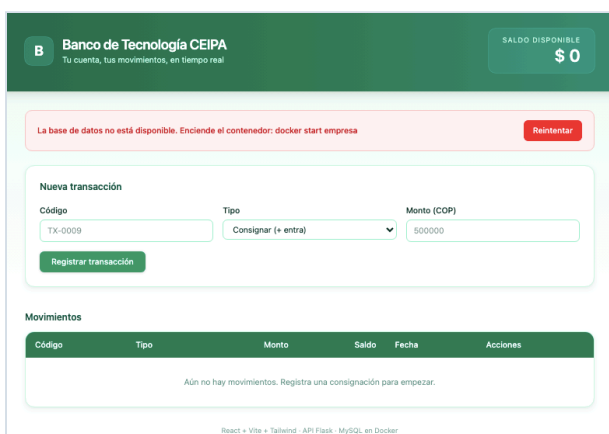


Figura 16. Con el contenedor detenido, la interfaz muestra el mensaje de la API y el botón «Reintentar».

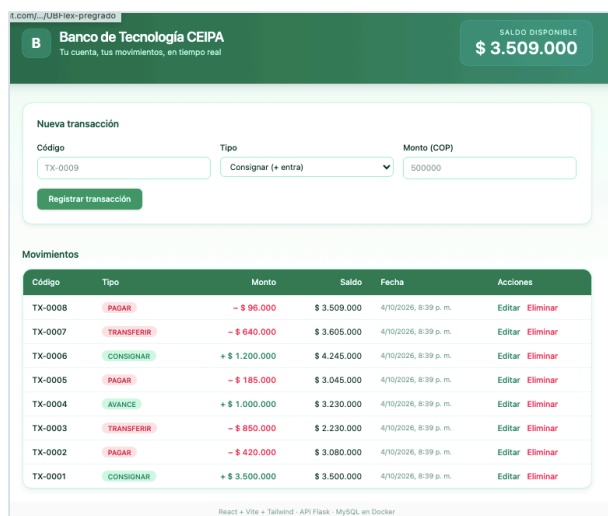


Figura 17. Con el contenedor encendido de nuevo, «Reintentar» recupera el saldo de \$3.509.000 y los movimientos.

Organización del repositorio público

El código se versionó con Git y se publicó en github.com/DavisZulu/Estacion_3. El archivo `.env`, que contiene la contraseña de la base de datos, y las carpetas de dependencias (`node_modules`, `__pycache__`) quedan excluidos mediante `.gitignore`; en su lugar se entrega `.env.example` como plantilla.

RUTA

CONTENIDO

BACKEND/app.py	Arranque de Flask, ruta <code>/api/salud</code> y respuesta 503 cuando MySQL no está disponible.
BACKEND/schema.prisma	Modelo de datos: tabla <code>Transaccion</code> con id, código, tipo, monto y fecha.
BACKEND/routes/	Capa 1: rutas de la API.
BACKEND/controllers/	Capa 2: validaciones, reglas del banco y cálculo del saldo.
BACKEND/models/	Capa 3: consultas a la base de datos con Prisma.
BACKEND/probar_api.py	Prueba automática del CRUD y carga de datos de ejemplo.
FRONTEND/src/pages/Transacciones.jsx	Pantalla del banco.
FRONTEND/src/api/transacciones.js	Llamadas a la API y manejo de errores de conexión.
PORTAFOLIO_WEB/	Código fuente del portafolio profesional (sección 08).
docs/ y capturas/	Diagramas de arquitectura y de clases, y evidencias de la ejecución.
README.md	Descripción del sistema, reglas del banco, instrucciones de ejecución en macOS y tabla de la API.

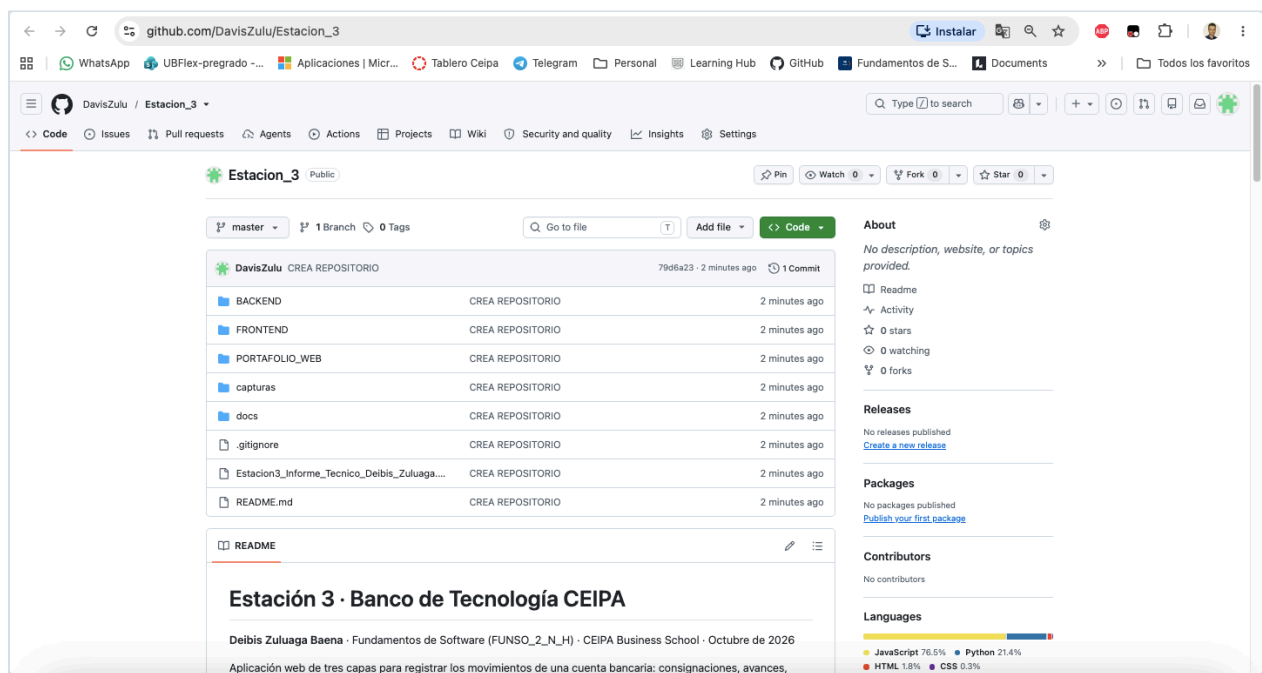


Figura 18. El repositorio público Estacion_3 en GitHub, con las carpetas del proyecto, el informe y el README.

Portafolio profesional

El proyecto se complementa con un portafolio profesional que presenta el perfil, las habilidades, los proyectos y la trayectoria del autor. El sitio está publicado en <https://davis-zuluaga.onrender.com> y su código fuente se incluye en la carpeta `PORTAFOLIO_WEB` del repositorio de esta entrega.

SITIO PUBLICADO

<https://davis-zuluaga.onrender.com>

Código fuente: github.com/DavisZulu/PORTAFOLIO_WEB y carpeta `PORTAFOLIO_WEB` de este repositorio.

ASPECTO

DETALLE

Tecnologías	React 18, Vite 5 y Tailwind CSS 3. Se publica como sitio estático, sin servidor de aplicación.
Aplicación instalable (PWA)	Con <code>vite-plugin-pwa</code> se generan el <i>manifest</i> y el <i>service worker</i> : el sitio puede instalarse en el computador o en el celular como una aplicación, con ícono propio.
Contenido separado del diseño	Todo el contenido vive en <code>src/data/portfolioData.js</code> . Los componentes de <code>src/components/</code> solo leen esos datos: actualizar el portafolio no requiere modificar ningún componente. Es el mismo principio de responsabilidad única aplicado a la interfaz.
Secciones	Inicio, Sobre mí, Habilidades, Proyectos, Experiencia y educación (línea de tiempo) y Contacto.
Experiencia de uso	Diseño adaptable a celular, modo claro y oscuro según la preferencia del sistema o del usuario, y navegación por secciones.
Publicación	Render, como sitio estático.
Copia en esta entrega	<code>Estacion_3/PORTAFOLIO_WEB</code> , sin <code>node_modules</code> ni archivos generados; se ejecuta con <code>npm install</code> y <code>npm run dev</code> .

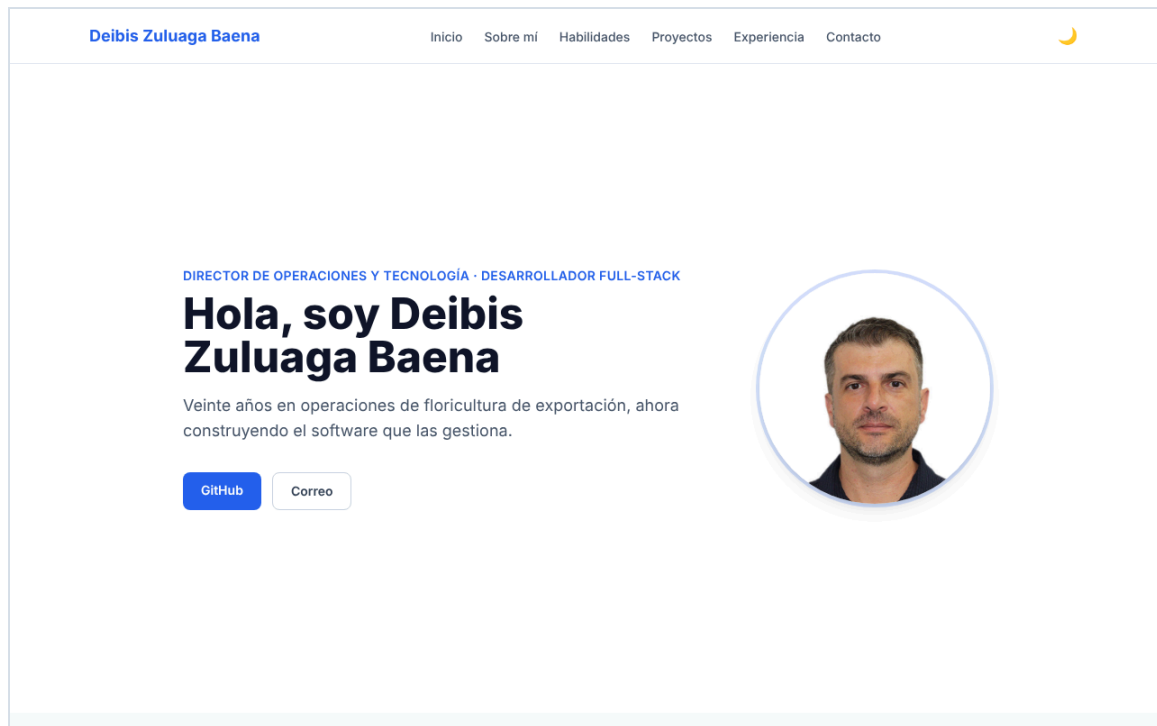


Figura 19. Página de inicio del portafolio, ejecutado desde la carpeta `PORTAFOLIO_WEB` incluida en el repositorio.

Secciones, modo oscuro y versión móvil

RELACIÓN CON EL PROYECTO

Quantum Core es la primera pieza de software con arquitectura completa: interfaz, API, base de datos en contenedor, pruebas y repositorio público. El portafolio es el lugar donde ese trabajo se presenta de forma profesional, y el repositorio de esta entrega reúne ambos para que puedan revisarse juntos.

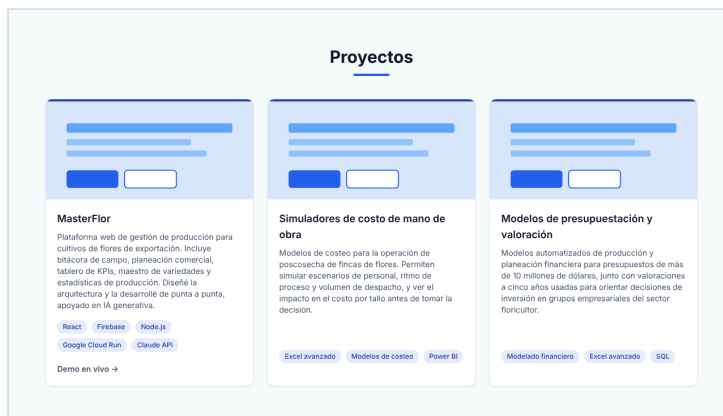


Figura 20. Sección de proyectos. Cada tarjeta se genera a partir de un objeto de `projectsData`.

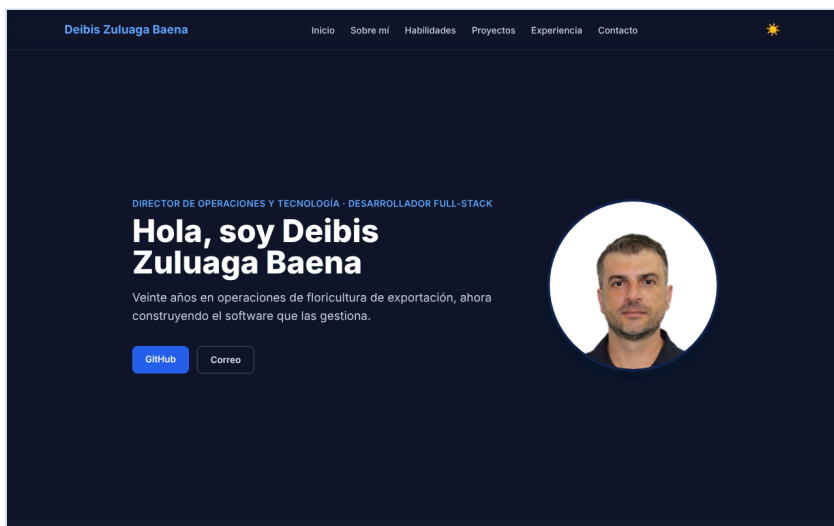


Figura 21. Modo oscuro, aplicado según la preferencia del sistema o con el botón de la barra superior.



Figura 22. Versión para celular.

Conclusiones

El proyecto demostró que tres procesos independientes pueden trabajar como una sola aplicación mediante contratos simples: HTTP con JSON entre la interfaz y la API, y Prisma entre la API y la base de datos. La separación en capas no solo ordena el código; fue lo que permitió transformar el CRUD de transacciones en una cuenta bancaria sin rediseñar el sistema: las rutas y el acceso a datos se mantuvieron, y los cambios se concentraron en el controller y en la interfaz.

La revisión del código original mostró que un sistema puede funcionar en el recorrido normal y aun así responder mal en los casos límite: confirmar una eliminación que no ocurrió o aceptar un valor que no corresponde a ninguna regla. La transacción T001, con un impacto digitado a mano, es la evidencia concreta de por qué las reglas del negocio deben vivir en el servidor y no en el formulario.

Calcular el saldo a partir de los movimientos, en lugar de almacenarlo, resolvió el problema de raíz: el saldo siempre coincide con lo registrado y la misma función permite rechazar cualquier creación, edición o eliminación que deje la cuenta en negativo. La validación de los datos de entrada y los códigos de respuesta precisos completan ese control, de modo que la interfaz puede explicar al usuario qué ocurrió en cada caso.

La verificación con el script de pruebas confirmó, en una sola ejecución, el CRUD completo y las reglas del banco sobre la base de datos real, y la demostración en el navegador mostró el sistema funcionando de principio a fin. Con el repositorio público y el portafolio publicado, el proyecto queda como evidencia verificable del trabajo realizado.

Referencias

- Docker Inc. (2026). *Docker documentation: docker container run*. <https://docs.docker.com/reference/cli/docker/container/run/>
- Fielding, R. T., Nottingham, M., & Reschke, J. (Eds.). (2022). *HTTP semantics* (RFC 9110). Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc9110>
- Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.
- Meta Platforms. (2026). *React documentation*. <https://react.dev/>
- Pallets. (2026). *Flask documentation*. <https://flask.palletsprojects.com/>
- Prisma Client Python. (2026). *Prisma Client Python documentation*. <https://prisma-client-py.readthedocs.io/>